Basic RAG vs. Agentic RAG Implementation with LlamaIndex

Explore how to implement both a traditional RAG pipeline and an advanced agentic RAG system using LlamaIndex. Learn to promote retrieval pipelines as tools for intelligent agents, enabling flexible, scalable AI research assistants with self-directed reasoning.

In our previous lessons, we built a solid theoretical foundation. We deconstructed the agent into its core components and explored the sophisticated ReAct reasoning strategy that brings it to life.

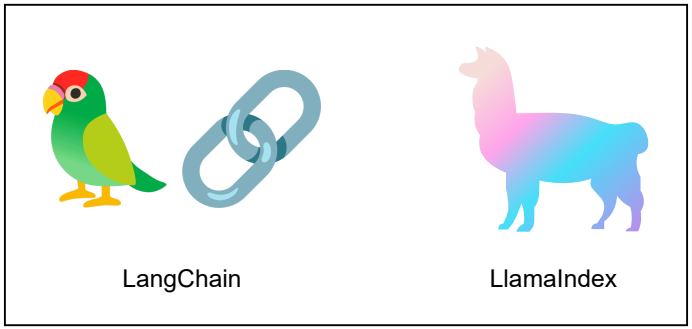
Now, it's time to bridge the gap between theory and practice. In this lesson, we will get our hands on the keyboard and write the code for our very first agentic RAG system. By implementing a standard RAG pipeline and an agentic system side by side, we will focus on the crucial differences in their architecture and code, understanding how the agentic approach provides a more flexible and scalable foundation.

An overview of agentic frameworks

Before we write our first line of code, it's helpful to understand the landscape. Building an agent from scra would require us to manually handle complex prompt templating, LLM response parsing, and the logic for the reasoning loop. Fortunately, several powerful open-source frameworks do this heavy lifting for us.

Ask

The two most prominent frameworks in the Python ecosystem are:



Agent implementation frameworks

- **LangChain:** One of the earliest and most popular frameworks for building LLM applications. For creating sophisticated agents, its companion library, **LangGraph**, is typically used. LangGraph allows developers to define agentic workflows as stateful graphs, enabling complex cycles, branching, and human-in-the-loop interactions.
- **LlamaIndex:** While it also has broad agentic capabilities, LlamaIndex has its roots specifically in optimizing RAG pipelines. It provides highly advanced and easy-to-use components for data ingestion, indexing, and retrieval. Its agentic layer is designed to integrate seamlessly with these powerful RAG components.

> If you're interested in exploring these frameworks, you might find dedicated courses on LlamaIndex and LangChain with LangGraph helpful.

Ask

For this course, we will be using LlamaIndex. Its data-centric focus makes it the perfect choice for mastering agentic RAG. It allows us to build state-of-the-art retrieval pipelines and then elegantly layer agentic reasoning

on top of them.

Essential LlamaIndex syntax

To get started, let's familiarize ourselves with two core LlamaIndex concepts that we will be using.

- **The query engine:** This is the object that represents our complete, runnable RAG pipeline. We will use a `VectorStoreIndex` to create a `query_engine`, and we can interact with it directly using `query_engine.query("Your question")`. A query engine encapsulates the entire RAG process, handling both the retrieval of information from the index and the final generation of the answer.
- **The tool:** In LlamaIndex, a `Tool` is a wrapper that gives an agent a specific capability. We will wrap our RAG `QueryEngine` in a `QueryEngineTool`. The most important part of a `Tool` is its `description`, a natural language string that tells the agent what the tool is for. The agent's reasoning ability is highly dependent on how well we write these descriptions.

Create our custom data for examples

To truly see the power and limitations of these systems, we need to use data that the LLM has never seen before. This guarantees that the answers are coming from our retrieval process, not the model's internal knowledge. Let's create our own unique knowledge base.

We create a data directory, and inside that we create two new text files.

Ask

1. `project_cygnus_overview.txt`:

```
Project Cygnus is a next-generation, AI-powered data analytics platform developed by the fictional
```

```
company "Innovate Inc." The project lead is Dr. Aris Thorne. The primary goal of Project Cygnus is
to achieve a beta release by Q4 2025, targeting enterprise-level financial institutions.
```

project_cygnus_overview.txt

2. `project_cygnus_technical_spec.txt`:

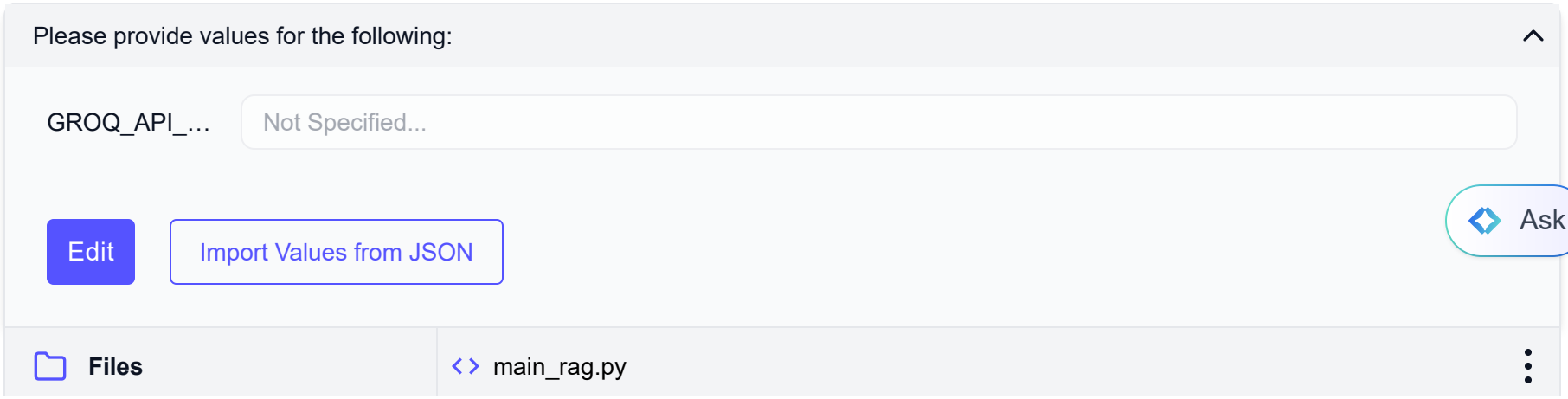
```
The backend of Project Cygnus is built on a microservices architecture using Python and Rust.
For data processing, it leverages a distributed graph database and a real-time event streaming
pipeline using Apache Kafka. The frontend is a progressive web app built with React.
```

project_cygnus_technical_spec.txt

Now, let's use this data in our code.

Implementing and testing the static RAG pipeline

Let's start by building our baseline, a straightforward, linear RAG system. This represents the traditional approach to RAG, where the application's sole purpose is to retrieve information and generate an answer.



main_rag.py

data

```python
import os
from dotenv import load_dotenv
from llama_index.core import (
    VectorStoreIndex,
    SimpleDirectoryReader,
    Settings,
)
from llama_index.embeddings.ollama import OllamaEmbedding
from llama_index.llms.groq import Groq

# --- Load Environment Variables & Configure Settings ---
load_dotenv()
Settings.llm = Groq(model="llama-3.1-8b-instant", api_key="gsk_NLtN9XCRliL
Settings.embed_model = OllamaEmbedding(model_name="nomic-embed-text")

# --- Load Our Custom Data ---
print("📂 Loading custom documents about Project Cygnus...")
documents = SimpleDirectoryReader("./data").load_data()
print(f"✅ Loaded {len(documents)} document(s).")

# --- Create Index and Query Engine ---
print("📊 Creating index and query engine...")
index = VectorStoreIndex.from_documents(documents)
query_engine = index.as_query_engine(similarity_top_k=1)
print("✅ Basic RAG Pipeline is Ready.")
```

Saved

Ask

Terminal +

Did you find this helpful?

Implementation of a standard, static RAG pipeline using LlamaIndex

This code builds a complete, self-contained RAG system. The `query_engine` object encapsulates all the logic for retrieval and generation.

- **Lines 1-9:** These lines handle the initial setup by importing all the necessary modules and classes. We import standard libraries like `os` and `dotenv`, and core components from `llama_index` for building our RAG pipeline, including the `Groq` LLM and the `OllamaEmbedding` model.
- **Line 12:** `load_dotenv()` reads the `.env` file in our project directory and loads the `GROQ_API_KEY` as an environment variable, keeping our secret keys out of the source code.
- **Line 13:** We configure `Settings.llm` to use the `Groq` class, specifying the `llama-3.1-8b-instant` model and passing our API key. `Settings` is a global object in LlamaIndex that makes this LLM available to all other components, without needing to pass it around manually.
- **Line 14:** We configure `Settings.embed_model` to use a local embedding model, `nomic-embed-text`, running via Ollama. This model will be used to convert all our text chunks into numerical vectors.
- **Line 18:** `SimpleDirectoryReader("./data").load_data()` is a convenient LlamaIndex utility. It automatically finds all readable files inside the `./data` directory, loads their content, and converts them into a list of `Document` objects.
- **Line 23:** `index = VectorStoreIndex.from_documents(documents)` is the core of the indexing process. This single command tells LlamaIndex to take our loaded documents, and automatically split them into small text chunks. It also instructs the LlamaIndex to use the `embed_model` from `Settings` to create a vector for each chunk, and store them all in a simple, in-memory vector index.

Ask

- **Line 24:** `query_engine = index.as_query_engine(...)` converts our indexed data into a ready-to-use `QueryEngine` object. This object is our complete, runnable RAG pipeline. We set `similarity_top_k=1` to tell the retriever to only fetch the single most relevant text chunk for each query.
- **Line 28:** We call the `query_engine.query()` method with our question. This single call executes the entire online RAG workflow. It embeds the query, searches the vector index for the most similar chunk, creates a prompt containing that chunk and our question, and sends it to the `Groq` LLM to generate a final, grounded answer.
- **Line 32:** The final response object returned by the query engine is printed to the console.

The conceptual shift: From pipeline to tool

Now, we will upgrade our system to an agentic one. This requires a fundamental shift in our thinking.

We are not throwing away our RAG pipeline. We are promoting it. Instead of being the entire application, it will become the first tool that our intelligent agent can use.

Analogy: We've built a very good "book-reading machine" (our `query_engine`). Now, we're going to hire a "research assistant" (the agent) and give them that machine to use as one of their capabilities.

Now, let's build the agent. The crucial difference will be in the `description` of our tool. We must be very specific about what the tool is for.

Ask

Implementing the agentic RAG system

Let's see what this promotion looks like in code. Notice how we reuse the same core RAG components but assemble them in a more flexible, agent-integrated way.

Values for some API keys are not provided. Provide them here to use throughout the course. Otherwise disable this prompt by removing API key placeholders from code.

GROQ_...

Cancel | Clear All | Save Changes

Files

- main_rag.py
- data

main_rag.py

```
33  )
34
35  # --- 2. Create the Agent ---
36  print("Creating the ReAct Agent...")
37  agent = ReActAgent(
38      tools=[rag_tool],
39      llm=Settings.llm,
40      system_prompt = """You are a helpful assistant. Carefully
41      analyze the user's query and decide if one of your tools is
42      appropriate. Only use a tool if the user's question directly
43      relates to the tool's description.""",
44      verbose=True
45  )
46  print("✅ Agentic RAG System is Ready.")
47
48  # --- Test
49  import asyncio
50  async def run_agent():
51      for query in queries:
52          response = await agent.run(user_msg=query)
53          print(f"User: {query}\nAgent: {response}\n")
54  queries = [
```

Ask

```
    "Who is the lead of Project Cygnus?"
]
# Run the async function
asyncio.run(run_agent())
```

Saved

Run

Terminal +

Did you find this helpful?

Upgrading the RAG pipeline to an agentic system by wrapping the query engine as a tool for a ReActAgent to use

This code upgrades our previous static RAG pipeline into a dynamic, agentic system. The key change is that instead of being the entire application, the RAG pipeline is now just one tool that an intelligent agent can choose to use.

- **Lines 1-12:** We import the same libraries as before, but now we also import `QueryEngineTool` to wrap our RAG pipeline and `ReActAgent` which will be our agent's main class.
- **Lines 15-17:** This section remains the same. We load our API key from the `.env` file and use the `Settings` object to globally configure our `Groq` LLM and local `OllamaEmbedding` model.
- **Lines 20-22:** This code also remains unchanged. It loads our custom documents from the `./data` directory, creates a vector index from them, and then builds the `query_engine` object, which is our runnable RAG pipeline.
- **Lines 26-33:** This is a crucial new step where we "promote" our pipeline to a tool.
  - **Line 26:** We use the `QueryEngineTool.from_defaults()` class method to wrap our existing `query_engine`.

Ask

- **Line 27:** We pass our `query_engine` object, turning it into a modular capability.
- **Line 28:** We give the tool a simple, unique `name` for the agent to reference.
- **Lines 29-32:** We provide a detailed `description`. This is the most critical part for the agent. The agent's LLM brain will read this natural language description to understand what the tool does and decide when it is relevant to a user's query.

- **Lines 37-45:** Here, we assemble and create the agent itself.
  - **Line 37:** We instantiate the `ReActAgent` class.
  - **Line 38:** The `tools` parameter is a list of all capabilities that the agent has. For now, it only contains our single `rag_tool`.
  - **Line 39:** The `llm` parameter connects the agent to its "brain," the `Groq` model we configured in `Settings`.
  - **Lines 40-43:** The `system_prompt` provides the agent with its high-level instructions or "operating manual." It tells the agent how to analyze queries and make decisions about using its tools.
  - **Line 44:** `verbose=True` is an invaluable setting for learning and debugging. It instructs the agent to print its entire internal `Thought -> Act -> Observe` cycle to the console.
- **Lines 49-58:** This section sets up and runs the test.
  - **Line 49:** We import `asyncio`, Python's library for running asynchronous code. This is needed because agent actions, like API calls, are often asynchronous.
  - **Lines 50-53:** We define an `async def` function `run_agent()`. Inside, it loops through our test qu and calls `await agent.run()`. The `await` keyword is essential; it pauses the function until the agent finishes its potentially multi-step reasoning and tool-use cycle before proceeding.

Ask

- **Lines 54-56:** We define a list of `queries` to test our agent with.
- **Line 58:** `asyncio.run(run_agent())` starts the asynchronous event loop and executes our main test function.

> When we run the code above, we see the agent's raw output provides a real-world look at its internal process. While we won't see the literal words "Thought," "Act," and "Observe," the log clearly shows this cycle in action. We will deconstruct this trace to see the agent's intelligence at work below.

Deconstructing the agent's thought process

We can map the output log directly to the `Thought -> Act -> Observe` cycle we learned in the previous lesson.

The first thought (deciding to act)

The first time we see a request to Groq, the agent is performing its initial reasoning step.

```
HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
```

At this moment, the agent has analyzed our query ("Who is the lead of Project Cygnus?") and its system prompt. It has read the description of the `project_cygnus_rag_tool` and concluded that the tool is relevant. Its internal "thought" is to use the RAG tool.

The `act` (calling the RAG tool)

Next, the agent executes the tool. For our RAG tool, this action involves two distinct steps visible in the log:

Ask

```
HTTP Request: POST http://localhost:11434/api/embed ...
HTTP Request: POST https://api.groq.com/openai/v1/chat/completions ...
```

1. **Embedding the query:** The call to `localhost:11434` is the RAG tool using our local Ollama model to turn our question into a vector for searching.
2. **Generating the answer:** The second call to Groq is the `QueryEngine` part of the tool. It has retrieved the relevant text chunk from our documents and is now asking the LLM to synthesize an answer based on that context.

The result of this entire two-step action becomes the `Observation` that the agent will use in its next reasoning cycle.

The final thought and response

After the tool returns its result (the `Observation`), the agent starts another reasoning cycle.

```
HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
```

This final call to Groq is the agent looking at the `Observation` (which contains the answer “Dr. Aris Thorne...”) and thinking, “I have successfully found the answer. The task is complete. I can now give the final response to the user.” It then generates the clean, final output that we see.

Implementation differences and conclusion

Ask

On this simple, one-question task, both systems will give you the same correct answer. However, their underlying architectures and implementation are fundamentally different. The agentic approach introduces a layer of

abstraction that provides immense flexibility.

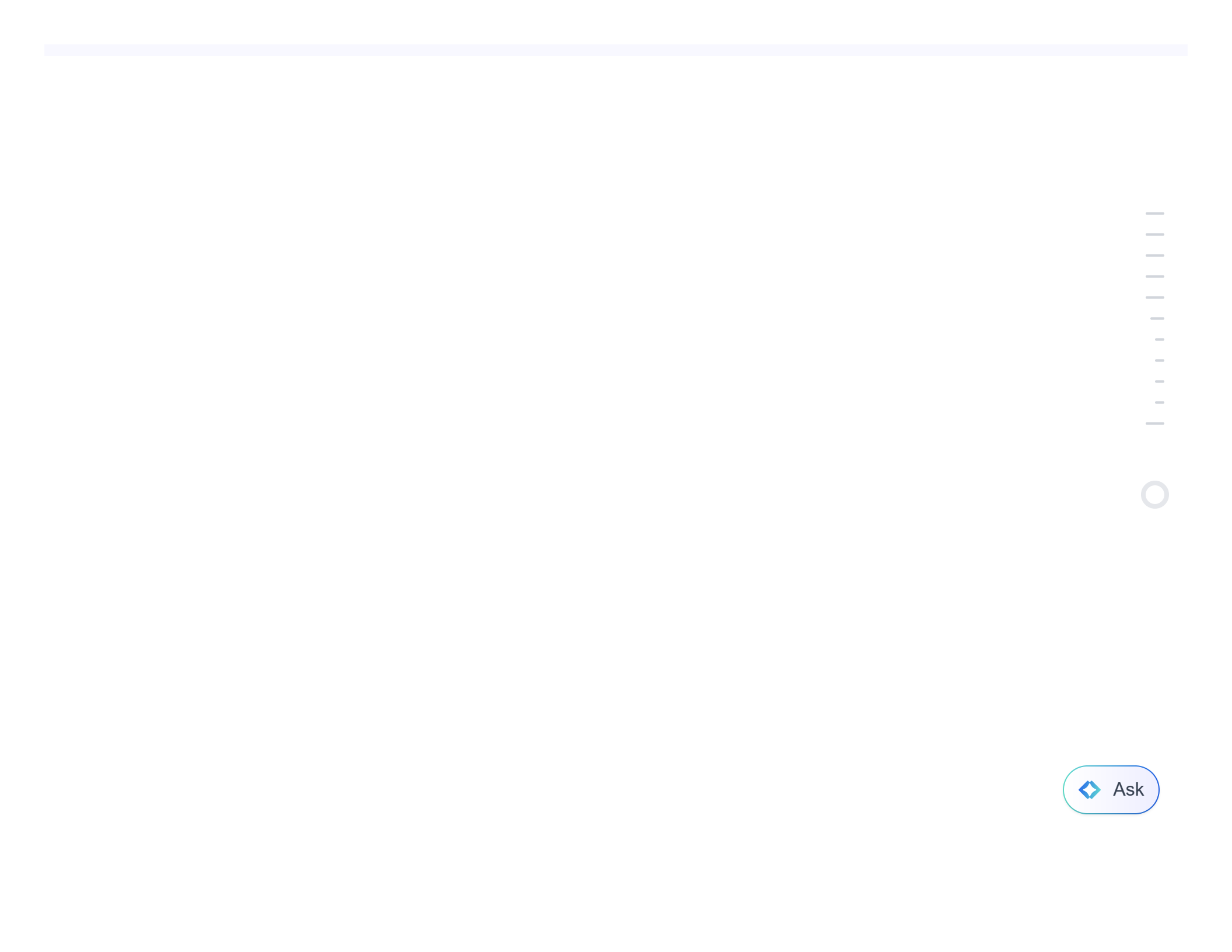
Let's summarize the key differences that we saw in the code:

Aspect	Static RAG Pipeline	Agentic RAG System
Core Object	A monolithic `QueryEngine`.	A `ReActAgent` that orchestrates a list of `Tools`.
Interaction	We call a single `.query()` method.	We interact conversationally using methods like `.run()`.
Flexibility	Inflexible. It can only perform RAG.	Highly flexible. The agent *decides* if RAG is needed.
Extensibility	Hard to extend. Adding a new capability (e.g., a calculator) would require rewriting the core logic.	Easy to extend. We can simply add a new `Tool` to the agent's `tools` list.

While the static pipeline is a single-purpose "book-reading machine," the agentic system is a "research assistant" that we've given a book-reading machine to. This distinction is the key to building more advanced systems.

In our next lesson, we will begin to design a more complex, multi-tool agent. It is there that the true power and scalability of this agentic architecture will become undeniable.

Ask


Ask